

初始布局设计

为了加快全局布局的收敛速度，在全局布局之前，进行了一个初始布局，为全局布局提供一个初始值，即初始布局给出可移动标准单元和宏块的初始位置。

初始化布局的方法有很多，它本身就是一个完整的布局方案，但由于仅仅是初始化，它不追求布局位置的准确性，所以往往采用算法简单，速度快的方法，如随机布局、聚类驱动布局、二次方程解析布局等，下面介绍这三种方法。

不同的方法具有不同的计算速度和效果，得到不同的线长（HPWL），通过设计和仿真，从连接保持能力、时间复杂度、面积控制、适用规模等方面，对各种方法进行比较分析。

1. 随机布局

这是最简单的一种布局，不存在算法，该方法是在布局区域内为每个可移动单元随机分配位置，确保随机位置在布局区域内，不考虑单元重叠。

2. 聚类驱动布局

聚类驱动布局（Clustering-Driven Placement）是一种层次化的布局方法，旨在通过“分而治之”的策略，应对大规模集成电路（VLSI）设计中布局问题的复杂性。核心思想是：不直接处理数量庞大的标准单元，而是先将它们聚类成多个“簇”（Cluster），每个簇视为一个质点，簇位置即是簇质心，再在簇内部安排单元。

参考算法步骤：

步骤 1：聚类成簇

步骤 1-1：初始化每个模块为一个独立的簇；

步骤 1-2：基于网表构建连接强度图（权重计算），即将电路的网表连接关系，转化为簇与簇之间的数值权重（连接越紧密，权重越大）；

步骤 1-3：迭代地选择权重最大的簇对，将二者合并成一个，更新新簇与其他簇的权重及其位置，直到停止条件（如簇的数目）

步骤 2：簇内布局

根据簇的面积和模块数量，在质心周围划分出一块矩形框，再把矩形框均匀切成 N 个小格子，最后把模块挨个放进格子里。

参考文献：PPA-Relevant Clustering-Driven Placement for Large-Scale VLSI Designs (DAC 2024)

3. 二次方程解析布局

初始布局忽略重叠约束，直接进行线长最小化。基本思想是用二次函数模拟线长公式（1），通过最小化二次函数，求得单元的位置，为后续全局布局优化提供一个初始布局。与全局布局不同，初始布局仅考虑线长，而没有考虑单元的重叠问题，即没有考虑布局密度问题。

3.1 目标模型

采用参考文献[1]的模型。

$$\Gamma = \frac{1}{2} \sum_{n \in \mathcal{N}} \sum_{e \in \mathcal{E}_n} w_{x,pq} (x_p^{\text{pin}} - x_q^{\text{pin}})^2 + w_{y,pq} (y_p^{\text{pin}} - y_q^{\text{pin}})^2 \quad (1)$$

$$= \sum_{n \in \mathcal{N}} \Gamma_{n,x} + \Gamma_{n,y} = \Gamma_x + \Gamma_y. \quad (2)$$

其中 x 部分：

$$\Gamma_{n,x} = \frac{1}{2} \sum_{p=1}^P \sum_{q=p+1}^P w_{x,pq} (x_p^{\text{pin}} - x_q^{\text{pin}})^2. \quad (3)$$

其中系数 w 为：

$$w_{x,pq} = \frac{1}{P-1} \frac{1}{|x_p^{\text{pin}} - x_q^{\text{pin}}|}. \quad (4)$$

其中 P 为 Pin 的总数。当前目标为最小化（1）式的模型。

3.2 目标最小值的求解推导

1) 单个连接

能量函数为：

$$E = w(x_1 - x_2)^2$$

w 为连接强度（weight），表示“两点间的线拉得多紧”，将括号展开：

$$E = w(x_1^2 - 2x_1x_2 + x_2^2)$$

把它写成矩阵形式，设位置向量为：

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

那么 E 可以重写为：

$$E = x^T \begin{bmatrix} w & -w \\ -w & w \end{bmatrix} x$$

这个矩阵正好体现了“自己对自己 +w，对邻居 -w”的关系。这也是线性代数里典型的拉普拉斯矩阵（Laplacian Matrix）形式。

2) 引入偏移量和所属节点坐标

需要注意的是，上述的 x 代表的是针脚（pin）的绝对坐标（absolute position），实际布局过程中，将节点的坐标当作未知数 x ，针脚相对于节点的偏移量命名为 $offset$ ：

$$E = w((x_1 + offset_1) - (x_2 + offset_2))^2$$

展开：

$$\begin{aligned} E &= wx_1^2 - 2wx_1x_2 + wx_2^2 + 2w(offset_1 - offset_2)x_1 + 2w(offset_2 - offset_1)x_2 + const \\ &= wx^T \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} x + 2w(offset_1 - offset_2)x^T \begin{bmatrix} 1 \\ -1 \end{bmatrix} + const \\ &= x^T Ax + x^T b + const \end{aligned}$$

其中：

A 是对称矩阵

b 是线性项

$const$ 是常数，对导数没有影响。

3) 考虑终端常量

由于终端位置固定，终端中心的横坐标不是变量，求解时，需考虑当前一对 pin 是否含有终端，如下 3 种情况考虑：

- i) p, q 均不是终端；
- ii) p 不是终端， q 是终端；
- iii) p 是终端， q 不是终端；

除此外，由于上面考虑的是单元中心坐标，实际应考虑 pin 相对于中心的偏移量。

i) p, q 均不是终端

求二次函数

$$f(x_1, x_2) = w((x_1 + offset_1) - (x_2 + offset_2))^2$$

写成矩阵形式，即

$$w((x_1 + offset_1) - (x_2 + offset_2))^2 = \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$$

得到：

- $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$
- $A = w \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$
- $\mathbf{b} = 2w(\text{offset}_1 - \text{offset}_2) \begin{bmatrix} 1 \\ -1 \end{bmatrix}$
- $c = w(\text{offset}_1 - \text{offset}_2)^2$

ii) p 不是终端, q 是终端

求二次函数

$$w((x_1 + \text{offset}_1) - (\text{const} + \text{offset}_2))^2$$

写成矩阵形式

$$w((x_1 + \text{offset}_1) - (\text{const} + \text{offset}_2))^2 = \mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$$

得到:

- $\mathbf{x} = [x_1]$
- $A = [w]$
- $\mathbf{b} = [2w(\text{offset}_1 - \text{offset}_2 - \text{const})]$
- $c = w(\text{offset}_1 - \text{offset}_2 - \text{const})^2$

iii) p 是终端, q 不是终端

求二次函数

$$w((\text{const} + \text{offset}_1) - (x_2 + \text{offset}_2))^2$$

写成矩阵形式

$$w((\text{const} + \text{offset}_1) - (x_2 + \text{offset}_2))^2 = \mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$$

得到:

- $\mathbf{x} = [x_2]$
- $A = [w]$
- $\mathbf{b} = [-2w(\text{const} + \text{offset}_1 - \text{offset}_2)]$
- $c = w(\text{const} + \text{offset}_1 - \text{offset}_2)^2$

4) 对整个目标的求解

将目标函数（3）展开为矩阵形式，如参考文献[1]中描述，将可移动单元的中心横坐标（宏块和标准单元）组成 $\mathbf{X}$ 矢量：

$$\mathbf{x} = (x_1, x_2, x_3, \dots, x_M)^T. \quad (14)$$

将模型的二次函数写成矩阵形式为：

$$\Gamma_x = \mathbf{x}^T \mathbf{C}_x \mathbf{x} + \mathbf{x}^T \mathbf{d}_x + \text{const}. \quad (15)$$

对矩阵形式的目标函数求导且令导数为 0，得到式子：

$$A_x \mathbf{x} = -\mathbf{b}_x / 2$$

$$A_y \mathbf{y} = -\mathbf{b}_y / 2$$

采用求解方程组方法，解方程组，就可以得到最小化线长情况下的单元中心的坐标。

3.3 求解方法

求解方程组方法很多，可以采用 Biconjugate Gradient Stabilized Method，稳定双共轭梯度法求解，该方法适合求解此类大型稀疏问题。因为一个网络中连接的单元数相对总单元数少，比较适合用 BiCGSTAB 方法求解。

BCGSTAB 是一种用于求解非对称线性方程组 $A\mathbf{x}=\mathbf{b}$ 的迭代算法，其中非对称线性方程组就是指其系数矩阵 A 不满足对称性（即 A 不等于 A^T ），它是对双共轭梯度法（BiCG）的改进，旨在克服 BiCG 方法中常见的不规则收敛行为。

3.3.1 数据结构设计

设有 M 个可移动单元，则 A 的维数为 $M \times M$ ，通常 M 很大，故 A 通常是一个规模巨大的矩阵，但它是稀疏的，即仅有很少元素不为零。因为具有连接关系的单元对才对应非零值，无连接关系的大部分单元对对应位置为 0。例如：根据前面推导，对 (p, q) 均不是终端的管脚对，设其位置分别为 (i, j) ，贡献四个值

- $A[i][i] += w$
- $A[i][j] -= w$
- $A[j][i] -= w$
- $A[j][j] += w$

工业界一般使用 CSR（Compressed Sparse Row）格式来存储稀疏矩阵。该格式只保存非零元素，抛弃大量零值，大幅节省内存，适合行方向遍历、矩阵-向量乘法。

CSR 用三个一维数组表示矩阵所有非零元素：

i. `values[]`: 存储矩阵里所有非零元素的值，按行优先顺序（从上到下、每行从左到右）排列。

ii. `rowPtr[]`: 标记每一行非零元素在 `values` 数组中的起始位置，设行号是基于 0 开始，即第 0、1、2...行

- `row_ptr[i]`: 第 i 行第一个非零元素在 `values` 里的下标

- `row_ptr[i+1]`: 第 i 行最后一个非零元素下标的后一个位置，即下行的起始位置。

- 因此第 i 行的非零元素区间: `[row_ptr[i], row_ptr[i+1])`

iii. `colIndex[]`: 每个 `values` 数组值属于哪一列

3.3.2 求解过程

主要的步骤如下:

第一步: 从 `*.scl` 文件读出网格信息，其外包范围即是 `CoreRegion`。一般用 `CoreRegion.ll` (lower left) 表示左下角，`CoreRegion.ur` (upper right) 表示右上角。从 `*.pl` 文件读出的可移动单元 `node` 和不可移动终端单元 `terminal` 的初始位置，注意，读出的位置为元件左下角位置，该位置即为设计所优化的目标。

例如: 对 `thin1, adaptec1` 等, `CoreRegion.ll=(459,459)`, `CoreRegion.ur=(11151,11139)`, 所有可移动单元 `node` 左下角位置为 `(0,0)`。

第二步: 将所有可移动单元 `node` 的中心移动到 `CoreRegion` 的中心。

例如: 对 `thin1, adaptec1` 等, 初始时, 将所有 `node` 的中心从 `(0,0)` 移到了 `CoreRegion` 的中心 `(5805,5799)` 处。

第三步: 分 3 种情况考虑, 构建系数矩阵 `A` 与 `b`。对每一对管脚 p, q , 在如下三种情况下, 分别求 `A` 与 `b`, 即填矩阵的相应成员项, 但重复填写某成员时, 数据进行累加:

i) p, q 均不是终端;

ii) p 不是终端, q 是终端;

iii) p 是终端, q 不是终端;

注意: 管脚的位置的确定方式, `*.scl` 文件中读入的偏移是相对于管脚所属元件中心点位置的偏移。

第四步: 解线性方程组:

$$A_x \mathbf{x} = -\mathbf{b}_x / 2$$

$$A_y \mathbf{y} = -\mathbf{b}_y / 2$$

采用求解方程组, 可得到最小化线长情况下的各单元中心的坐标。

注: (4) 式中的系数 w 中包含未知数 x , 开始得不到准确值, 所以解方程组需要多次迭代, 即第一次用 x 的一个初值求 w , 之后进行解方程组; 第一次迭代求

得的 x 的结果作为第二次迭代的输入，在第二次迭代中会同样先求 w ，之后又进行解方程组，如此反复，直至方程组解的误差达到足够小或达到规定的迭代次数。

在每次得到结果后，需要判断下本次计算得到的中心点是否合规，即通过中心点计算左下角和右上角位置是否超出了 `CoreRegion` 的边界，若超出，则设置为 `CoreRegion` 的边界值。参考代码如下：

```
if (!module->isFixed)
{
    if (x - 0.5 * module->width < coreRegion.ll.x)
    {
        x = coreRegion.ll.x + 0.5 * module->width + EPS;
    }
    if (x + 0.5 * module->width > coreRegion.ur.x)
    {
        x = coreRegion.ur.x - 0.5 * module->width - EPS;
    }
    if (y - 0.5 * module->height < coreRegion.ll.y)
    {
        y = coreRegion.ll.y + 0.5 * module->height + EPS;
    }
    if (y + 0.5 * module->height > coreRegion.ur.y)
    {
        y = coreRegion.ur.y - 0.5 * module->height - EPS;
    }
}
```

下面详细讲解第三步构建系数矩阵 A 与 b 与第四步解线性方程组。总体思路：`Triplet` 负责构建 A ，`VectorXf` 负责存储 b 、 x ，`BiCGSTAB` 负责求解。

(1) 构建 A

i) A 的定义

```
Eigen::SparseMatrix<float, Eigen::RowMajor>
```

`RowMajor` 指定

- Ax 运算按行访问
- CSR 格式

于是通常这样声明 A ：

```
// 构建 x, y 两个方向的方程
```

```
Eigen::SparseMatrix<float, Eigen::RowMajor> X_A(nodeCount,
nodeCount);
Eigen::SparseMatrix<float, Eigen::RowMajor> Y_A(nodeCount,
nodeCount);
```

X_A 对应 x 方向的方程, Y_A 对应 y 方向的方程。

采用两个稀疏矩阵分别求解的方式, 方便后续分别展开

ii) A 的填充

Eigen 通过三元组 Triplet 构造稀疏矩阵:

```
// 构建 Triplet 指令集
vector<Eigen::Triplet<float>> tripletListX;

// 对指令集进行添加, 这里仅供举例, 在实际操作中要注意分情况
tripletListX.push_back(Eigen::Triplet<float>(i, i, w));
tripletListX.push_back(Eigen::Triplet<float>(j, j, w));
tripletListX.push_back(Eigen::Triplet<float>(i, j, -w));
tripletListX.push_back(Eigen::Triplet<float>(j, i, -w));

...

// 通过 setFromTriplets 的 api 一次性输入指令, 建立矩阵
X_A.setFromTriplets(tripletListX.begin(), tripletListX.end());
```

其中:

- $\text{triplet}(i,j,w)$ 表示 “ $A[i][j] = w$ ” 的一个数据点
- 而最终通过 `setFromTriplets()` 一次性建立矩阵
- 构建稀疏矩阵最重要的是“先累加信息, 再统一压缩”, 直接 `insert` 会非常慢。

(2) 构建向量 x 与 b : 用 `VectorXf` 存储未知变量和线性项

在放置问题中, x 与 b 是两个长度为 `nodeCount` 的向量:

- **x :** 未知变量 (模块位置)
- **b :** 线性项 (来自 `offset` 差)

我们用 Eigen 的 `VectorXf` 来存储它们:


```
// VectorXf 代表动态长度向量 (X), 内部存储数据类型为 float (f)
Eigen::VectorXf X_x(nodeCount), Y_x(nodeCount);
Eigen::VectorXf X_b(nodeCount), Y_b(nodeCount);
```

(3) 构建未知量 x 的初值

将所有节点的初值均设为 **core region**（布局区域）的 **中心点** 而在布局过程中， x 的初值来自模块当前坐标：

```
X_x(i) = curNode->getCenter().x;
```

(4) 由偏移量 $offset$ 差构成构建 b

$$b_i = -w(offset_i - offset_j)$$

实现中就是对 $X_b[i]$ 、 $X_b[j]$ 不断累加。

(5) 使用求解器求解 $Ax=b$:

采用迭代求解方式，反复调用 **BiCGSTAB** 工具求解逼近值。因此，初始值影响了求解的速度甚至能否求解。而对放置问题而言， $Ax=b$ 的解具有强烈的“连续性”。在一次迭代更新之后，下次优化的最优解通常只与前一次有轻微差别，因为 **net** 权重、**offset** 只发生小幅变化。因此，上一轮布局的位置天然就是下一轮求解器最好的初始值，而中心点正是这个初始值最好的开端。

```
for (int i = 0;; i++) {
    // 构建 A, b, x
    createSparseMatrix(X_A, Y_A, X_x, Y_x, X_b, Y_b);

    // 调用 BiCGSTAB 工具，构建求解器
    BiCGSTAB<Eigen::SparseMatrix<float, Eigen::RowMajor>,
    IdentityPreconditioner> solver;

    // 设置最大迭代次数
    solver.setMaxIterations(100);

    // 准备求解
    solver.compute(X_A);
```

```

// 求解
X_x = solver.solveWithGuess(X_b, X_x);
xError = solver.error();

// y 方向同样操作
solver.compute(Y_A);
Y_x = solver.solveWithGuess(Y_b, Y_x);
yError = solver.error();

// 写入并更新位置
updateModuleLocation(X_x, Y_x);
HPWL = db->calcNetBoundPins();

// 截止条件：残差达到特定范围，或者达到最大迭代次数
if (fabs(xError) < target_error && fabs(yError) < target_error && i > 4) {
    break;
}
if (i >= maxIterationNumber) {
    break;
}
}

```

i. **compute(A): 准备求解**

```
solver.compute(X_A);
```

分析 A 的矩阵结构并进行预处理，以便为后续迭代做初始化

ii. **solveWithGuess(b, x0): 以“上一轮位置”为初值求解**

```
X_x = solver.solveWithGuess(X_b, X_x);
```

用当前的位置解作为下一步求解的初值，这样更接近真实解，迭代也会更快

使用 *solveWithGuess(b, x_{old})*，可以显著降低 BiCGSTAB 的迭代次数，使求解器在数十步内收敛，而不用从全 0 解开始慢慢“爬”。这是一种典型的渐进式优化策略，也是放置器能在百万级节

点下保持可计算性的关键之一。

iii. solver.error(): 查看收敛情况

```
xError = solver.error();
```

- error 越小说明解越稳定
- 放置器不需要极高精度，通常 $1e-4 \sim 1e-5$ 就够用了
- (6) 同单元对跳过
如果在遍历针脚的时候，发现针脚 p 和 q 对应的相同单元，需要跳过，由于同 module 的 pins 永远属于同一个变量 x_k，它们之间不存在独立自由度。
- (7) 求解稳定性考虑
在计算 $weight = 1 / |x_1 - x_2|$ 时，不能让分母无限逼近 0，如果 pin 离得特别近，那么：
 - $|x_1 - x_2|$ 接近 0
 - $weight = 1 / distance$ 会突然变成几千、几万
 - 这将导致 BiCGSTAB 迭代直接开始震荡、不收敛。

解决办法：设置最小距离差 MIN_DISTANCE。

```
distanceX = max(distanceX, MIN_DISTANCE);

// 详细代码格式，仅供参考
float distanceX = fabs(pin1->getAbsolutePos().x - pin2->getAbsolutePos().x);

if (float_greaterorequal(distanceX, MIN_DISTANCE)) {
    weightX = constant1 / distanceX;
} else {
    weightX = constant1 / MIN_DISTANCE;
}
```

以使得迭代条件稳定，也能够正常收敛。

参考文献

- [1] P. Spindler, U. Schlichtmann, and F. M. Johannes, “Kraftwerk2—A fast force-directed quadratic placement approach using an accurate net model,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 8, pp. 1398–1411, Aug. 2008.